DEVSYNC · DATABASE DESIGN

Entity Relationship Diagram (ERD)

Detailed ERD for the whole application + one specialised ERD for every Core Feature

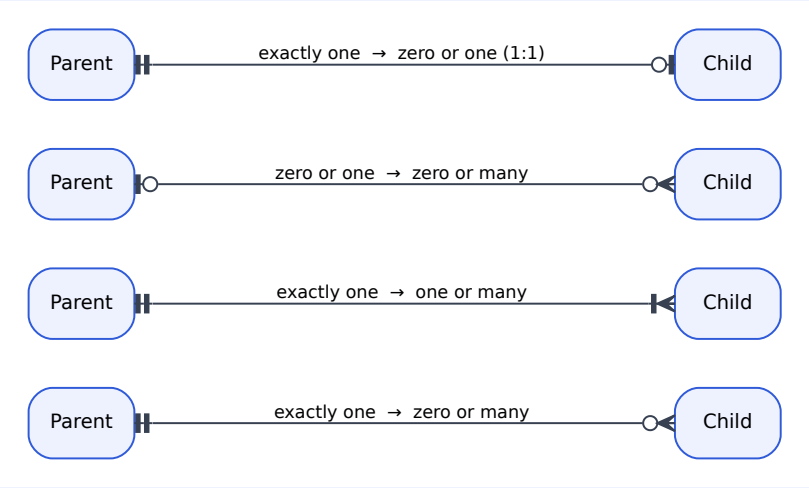
Project	**DevSync** — Secure AI Project Collaborator (Android · Flutter + Firebase)
Notation	Crow's Foot (Information Engineering)
Scope	29 entities · 52 relationships · 11 features
Sources	Core Features (core_features1.pdf) · DevSync Comprehensive Project Analysis (Requirements v1.2)
Companion file	DevSync_Logical_Physical_Schema.pdf
Date	29 October 2026

Contents

1. How to read the ERD 3
2. Entity Catalog 4
3. Relationship Catalog 5
4. Many-to-Many Resolution 8
5. Overview ERD — all entities and relationships 9
6. Detailed ERD — whole application (all attributes) 11
7. Feature ERDs (11 Core Features in scope) 13
    - Feature 1 — Splash & Onboarding 14
    - Feature 2 — Authentication (with Guest Mode) 15
    - Feature 3 — Profile (with Profile Completion) 17
    - Feature 4 — GitHub Integration 19
    - Feature 5 — AI Match 21
    - Feature 6 — Project Discovery 23
    - Feature 8 — Project Details 25
    - Feature 9 — Create Project 27
    - Feature 10 — My Teams 29
    - Feature 11 — My Projects 31
    - Feature 12 — Prompt Evaluation 33
8. Assumptions & Open Points 35

1. How to read the ERD

This ERD uses **Crow's Foot notation**. Every line between two entities is a relationship; the symbol at each end shows the cardinality (allowed number of rows) and the participation (mandatory or optional). **FK** attributes are orange, **PK** attributes are bold, and a **?** after a name means the attribute is **nullable**.



Symbol	Meaning	Participation
||	Exactly one	Total (mandatory)
o|	Zero or one	Partial (optional)
o<	Zero or many	Partial (optional)
|<	One or many	Total (mandatory)

Entity colours (modules)

Identity & Profile **GitHub Integration** **Projects & Teams** **AI, Matching & Evaluation** **Collaboration & Workspace** **Messaging**

Grey entities in the feature diagrams are **context entities**: they are shown with their PK only to illustrate links to other features; their full definition is in the ERD of their own feature.

Convention	Rule
Entity names	UPPER_SNAKE_CASE (singular). The physical table / collection name is shown in the catalog.
Attribute names	snake_case. **PK** primary key · **FK** foreign key · **UK** unique key · **?** nullable.
M:N relationships	Always resolved with an associative entity (see section 4), never drawn directly.
Derived values	Counters, completion percentage and similar derived values are **not** attributes in the ERD (they appear as denormalized fields in the physical schema file).
Guest	Not an entity: a guest has no account and nothing is stored.

2. Entity Catalog

Every entity with a short description, the features that use it, and where it comes from in the two source documents.

#	Entity / table	Module	Description	PK	Features	Source in documents
1	**USER** `users`	Identity & Profile	A developer / project owner account: account data and the public profile. A Guest is not stored here (no account, nothing persisted).	`uid`	2, 3	Core §2, §3 · FR-01, FR-03 · US-01, US-02 · UC-01
2	**USER_DEVICE** `user_devices`	Identity & Profile	User devices and their FCM token, used to deliver push notifications (one or more devices per user).	`device_id`	2	Core §10 op.12, §11 op.13 · FR-17 · US-16 · NFR-02
3	**SKILL** `skills`	Identity & Profile	Skill catalogue (programming languages, technical skills, soft skills, interests) shared by profiles and projects.	`skill_id`	3	Core §3, §9 · FR-03, FR-12
4	**USER_SKILL** `user_skills`	Identity & Profile	Associative table (M:N) between users and their confirmed skills. Each programming language carries a level.	`user_id, skill_id`	3	Core §3 (Programming Languages with level, Skills, Soft skills, Interests) · FR-03
5	**GITHUB_DATA** `github_data`	GitHub Integration	GitHub connection summary for a user (1:1). The OAuth token is NOT stored here (NFR-07).	`user_id`	4	Core §4 (users/{uid}/githubData) · FR-04 · US-03 · UC-01
6	**GITHUB_REPO** `github_repos`	GitHub Integration	Repository summaries pulled from GitHub (top languages are computed from them).	`repo_id`	4	Core §4 op.2 · FR-04
7	**DETECTED_SKILL** `detected_skills`	GitHub Integration	A skill inferred by the AI from GitHub. It is NOT a verified fact until the user accepts it (status).	`detected_id`	4	Core §4 (users/{uid}/detectedSkills) · FR-05 · US-04
8	**SKILL_EVIDENCE** `skill_evidence`	GitHub Integration	Evidence backing an inferred skill (repo / commits / PR). Resolves the multi-valued attribute.	`evidence_id`	4	Core §4 op.3 (evidence & confidence)
9	**PROJECT** `projects`	Projects & Teams	A project (created manually or AI-assisted). Project Purpose is mandatory and drives related fields.	`project_id`	6, 8, 9	Core §6, §8, §9, §11 · FR-12..FR-14 · US-10..US-13 · UC-04
10	**PROJECT_SKILL** `project_skills`	Projects & Teams	Associative table (M:N): the project's tech stack and required skills.	`project_id, skill_id`	6, 8, 9	Core §9 (Tech Stack and Required Skills) · FR-12
11	**PROJECT_ROLE** `project_roles`	Projects & Teams	Roles a project needs (Frontend, Backend, QA ...). AI-generated and editable.	`role_id`	6, 8, 9	Core §9 (Role Generator), §10 (Missing Role Alert) · FR-13
12	**ROLE_SKILL** `role_skills`	Projects & Teams	Associative table: skills required by each role (supports Missing Role Alert and Skill Gap). Design addition.	`role_id, skill_id`	9	Core §9 op.8, §10 op.10 (design addition)
13	**PROJECT_MEMBER** `project_members`	Projects & Teams	Team membership (associative between user and project). The owner is recorded as an Owner member.	`project_id, user_id`	9, 10, 11	Core §10, §11 · FR-15, FR-16 · US-14, US-15 · UC-03
14	**SAVED_PROJECT** `saved_projects`	Projects & Teams	Saved projects. Save and Favorites are merged into a single Saved list.	`user_id, project_id`	6, 8	Core §6 op.8, op.10 + Notes · FR-21, FR-22 · US-21, US-22
15	**INTEREST** `interests`	Projects & Teams	Express Interest: an action separate from Save and Request to Join. At most once per user and project.	`user_id, project_id`	6, 8	Core §6 op.9, §8 op.6 + Notes
16	**JOIN_REQUEST** `join_requests`	Collaboration & Workspace	A request to join a project. Sent once only (UNIQUE) with status Pending / Accepted / Rejected.	`request_id`	8, 11	Core §8, §11 · FR-15 · US-14 · UC-03
17	**INVITATION** `invitations`	Collaboration & Workspace	An invitation sent to a developer (Invite Developer). The owner can cancel it.	`invitation_id`	11	Core §11 op.9-11 · FR-16 · US-17

18	**PROMPT_VERSION** `prompt_versions`	AI, Matching & Evaluation	Prompt versions (v1, v2 ...) per generation type. The rule-based fallback is also recorded as a version.	`prompt_version_id`	12	Core §12 op.2-3, op.7 · US-25
19	**AI_GENERATION** `ai_generations`	AI, Matching & Evaluation	Log of every AI output (recommendation, project setup, pitch). Links an output to its prompt version.	`generation_id`	5, 12	Core §12 op.1 (Evaluate AI Suggestion), §5 op.1 (design addition)
20	**MATCH_RECOMMENDATION** `match_recommendations`	AI, Matching & Evaluation	A project recommendation for a user plus its explanation. All rows together form the Match History.	`match_id`	5	Core §5 · FR-07..FR-09 · US-06..US-08 · UC-02
21	**MATCH_SKILL** `match_skills`	AI, Matching & Evaluation	Per-recommendation detail: matched / complementary / close / missing skills, with a short learning tip for missing ones.	`match_id, skill_id`	5	Core §5 op.5-7 (Cross-skill, Skill gap, learning tip) · US-09
22	**AI_FEEDBACK** `ai_feedback`	AI, Matching & Evaluation	User rating of an AI output: thumbs up/down + clarity 1-5 + optional reason.	`feedback_id`	5, 12	Core §5 (Feedback bottom sheet), §12 · US-25
23	**PROMPT_EVAL_RUN** `prompt_eval_runs`	AI, Matching & Evaluation	Result of running a prompt version against a fixed test set (the required delivery evidence).	`run_id`	12	Core §12 op.5-6 · US-25
24	**TASK** `tasks`	Collaboration & Workspace	A task on the board: To Do / In Progress / In Review / Done.	`task_id`	10	Core §10 · FR-18 · US-18 · UC-05
25	**CONVERSATION** `conversations`	Messaging	A direct (1-to-1) conversation or a team conversation tied to a project.	`conversation_id`	10	Core §6 op.13 (Start chat), §10 (Team Chat) · FR-19, FR-20 · US-19, US-20
26	**CONVERSATION_PARTICIPANT** `conversation_participants`	Messaging	Associative table (M:N) between conversations and users, plus last-read time.	`conversation_id, user_id`	10	Core §6 op.13, §10 op.4
27	**MESSAGE** `messages`	Messaging	A message inside a conversation (real-time).	`message_id`	10	Core §10 op.4 · FR-19, FR-20
28	**NOTIFICATION** `notifications`	Collaboration & Workspace	An in-app notification (Today / Earlier list) with a reference for FCM and the deep link.	`notification_id`	10, 11	Core §10 op.12, §11 op.13 · FR-17 · US-16 · NFR-02
29	**AI_USAGE_COUNTER** `ai_usage_counters`	AI, Matching & Evaluation	Daily counter of AI requests per user (cost control).	`user_id, usage_date`	5	Risk R5 (cap AI requests per user) · NFR-01

3. Relationship Catalog

Every relationship with its type (1:1 or 1:N), the participation of both sides (Total = mandatory, Partial = optional) and the foreign key that implements it. All M:N relationships are resolved with associative entities (section 4).

#	Parent (one side)	Child	Type	Cardinality	Participation (parent / child)	Verb	Foreign key	Business rule
1	**USER**	**USER_DEVICE**	1:N	1 → 0..*	Partial / Total	registers	`USER_DEVICE.user_id`	A user can register several devices for FCM notifications
2	**USER**	**USER_SKILL**	1:N	1 → 0..*	Partial / Total	declares	`USER_SKILL.user_id`	The user's confirmed skills
3	**SKILL**	**USER_SKILL**	1:N	1 → 0..*	Partial / Total	is held in	`USER_SKILL.skill_id`	A skill can belong to many users
4	**USER**	**GITHUB_DATA**	1:1	1 → 0..1	Partial / Total	links	`GITHUB_DATA.user_id`	Connecting GitHub is optional (1:1) and can be disconnected

5	**GITHUB_DATA**	**GITHUB_REPO**	1:N	1 → 0..*	Partial / Total	summarises	`GITHUB_REPO.user_id`	Repository summaries
6	**USER**	**DETECTED_SKILL**	1:N	1 → 0..*	Partial / Total	is analysed for	`DETECTED_SKILL.user_id`	AI-inferred skills
7	**SKILL**	**DETECTED_SKILL**	1:N	1 → 0..*	Partial / Total	is detected as	`DETECTED_SKILL.skill_id`	Link to the catalogue
8	**DETECTED_SKILL**	**SKILL_EVIDENCE**	1:N	1 → 1..*	Total / Total	is backed by	`SKILL_EVIDENCE.detected_id`	Every inferred skill needs at least one piece of evidence
9	**GITHUB_REPO**	**SKILL_EVIDENCE**	1:N	0..1 → 0..*	Partial / Partial	evidences	`SKILL_EVIDENCE.repo_id`	Evidence may point to a specific repo (optional)
10	**USER**	**PROJECT**	1:N	1 → 0..*	Partial / Total	owns	`PROJECT.owner_id`	Every project has exactly one owner
11	**PROJECT**	**PROJECT_SKILL**	1:N	1 → 0..*	Partial / Total	requires	`PROJECT_SKILL.project_id`	Tech stack and required skills
12	**SKILL**	**PROJECT_SKILL**	1:N	1 → 0..*	Partial / Total	is required by	`PROJECT_SKILL.skill_id`	
13	**PROJECT**	**PROJECT_ROLE**	1:N	1 → 0..*	Partial / Total	defines	`PROJECT_ROLE.project_id`	Required roles (a Draft may have none yet)
14	**PROJECT_ROLE**	**ROLE_SKILL**	1:N	1 → 0..*	Partial / Total	needs	`ROLE_SKILL.role_id`	
15	**SKILL**	**ROLE_SKILL**	1:N	1 → 0..*	Partial / Total	is needed by	`ROLE_SKILL.skill_id`	
16	**PROJECT**	**PROJECT_MEMBER**	1:N	1 → 1..*	Total / Total	has	`PROJECT_MEMBER.project_id`	The owner is always a member, so a project has at least one member
17	**USER**	**PROJECT_MEMBER**	1:N	1 → 0..*	Partial / Total	joins as	`PROJECT_MEMBER.user_id`	
18	**PROJECT_ROLE**	**PROJECT_MEMBER**	1:N	0..1 → 0..*	Partial / Partial	is filled by	`PROJECT_MEMBER.role_id`	A member may have no specific role
19	**USER**	**SAVED_PROJECT**	1:N	1 → 0..*	Partial / Total	saves	`SAVED_PROJECT.user_id`	Save and Favorites are merged
20	**PROJECT**	**SAVED_PROJECT**	1:N	1 → 0..*	Partial / Total	is saved in	`SAVED_PROJECT.project_id`	
21	**USER**	**INTEREST**	1:N	1 → 0..*	Partial / Total	expresses	`INTEREST.user_id`	Express Interest is a separate action
22	**PROJECT**	**INTEREST**	1:N	1 → 0..*	Partial / Total	attracts	`INTEREST.project_id`	
23	**PROJECT**	**JOIN_REQUEST**	1:N	1 → 0..*	Partial / Total	receives	`JOIN_REQUEST.project_id`	
24	**USER**	**JOIN_REQUEST**	1:N	1 → 0..*	Partial / Total	submits	`JOIN_REQUEST.applicant_id`	Applicant
25	**PROJECT_ROLE**	**JOIN_REQUEST**	1:N	1 → 0..*	Partial / Total	is requested in	`JOIN_REQUEST.role_id`	The requested role is mandatory
26	**AI_GENERATION**	**JOIN_REQUEST**	1:1	0..1 → 0..1	Partial / Partial	drafts pitch of	`JOIN_REQUEST.pitch_generation_id`	The AI-generated Collaboration Pitch
27	**PROJECT**	**INVITATION**	1:N	1 → 0..*	Partial / Total	sends	`INVITATION.project_id`	
28	**USER**	**INVITATION**	1:N	1 → 0..*	Partial / Total	is invited by	`INVITATION.invitee_id`	Invited developer
29	**USER**	**INVITATION**	1:N	1 → 0..*	Partial / Total	issues	`INVITATION.inviter_id`	Owner who sent the invitation
30	**PROJECT_ROLE**	**INVITATION**	1:N	0..1 → 0..*	Partial / Partial	targets	`INVITATION.role_id`	
31	**AI_GENERATION**	**PROJECT**	1:1	0..1 → 0..1	Partial / Partial	sets up	`PROJECT.setup_generation_id`	AI-assisted project setup
32	**PROMPT_VERSION**	**AI_GENERATION**	1:N	1 → 0..*	Partial / Total	produces	`AI_GENERATION.prompt_version_id`	Every output is tied to a prompt version
33	**USER**	**AI_GENERATION**	1:N	1 → 0..*	Partial / Total	requests	`AI_GENERATION.user_id`	

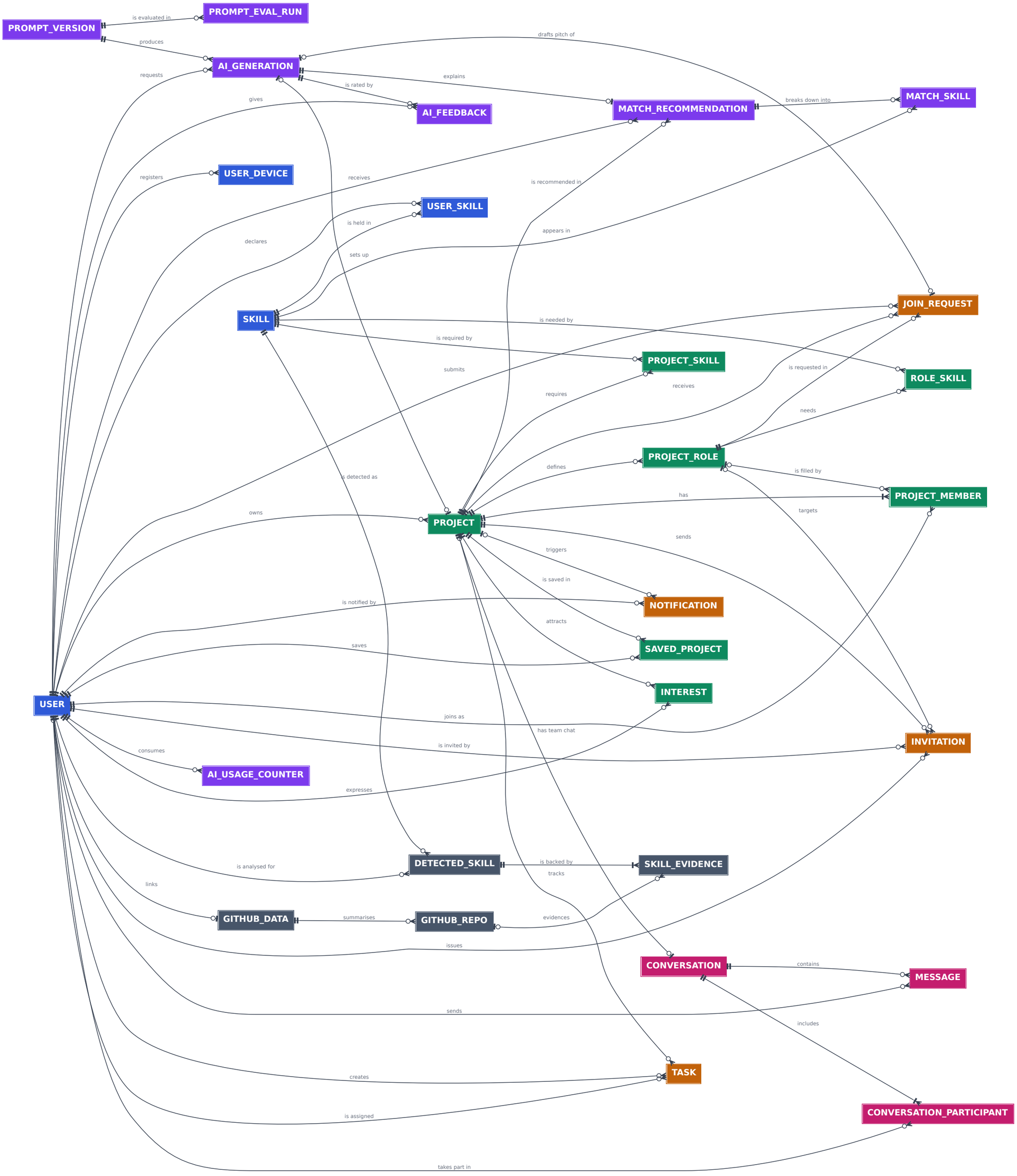
34	**PROMPT_VERSION**	**PROMPT_EVAL_RUN**	1:N	1 → 0..*	Partial / Total	is evaluated in	`PROMPT_EVAL_RUN.prompt_version_id`	Versions are compared on a fixed test set
35	**AI_GENERATION**	**MATCH_RECOMMENDATION**	1:1	1 → 0..1	Partial / Total	explains	`MATCH_RECOMMENDATION.generation_id`	
36	**USER**	**MATCH_RECOMMENDATION**	1:N	1 → 0..*	Partial / Total	receives	`MATCH_RECOMMENDATION.user_id`	Match History
37	**PROJECT**	**MATCH_RECOMMENDATION**	1:N	1 → 0..*	Partial / Total	is recommended in	`MATCH_RECOMMENDATION.project_id`	
38	**MATCH_RECOMMENDATION**	**MATCH_SKILL**	1:N	1 → 0..*	Partial / Total	breaks down into	`MATCH_SKILL.match_id`	
39	**SKILL**	**MATCH_SKILL**	1:N	1 → 0..*	Partial / Total	appears in	`MATCH_SKILL.skill_id`	
40	**AI_GENERATION**	**AI_FEEDBACK**	1:N	1 → 0..*	Partial / Total	is rated by	`AI_FEEDBACK.generation_id`	
41	**USER**	**AI_FEEDBACK**	1:N	1 → 0..*	Partial / Total	gives	`AI_FEEDBACK.user_id`	
42	**PROJECT**	**TASK**	1:N	1 → 0..*	Partial / Total	tracks	`TASK.project_id`	
43	**USER**	**TASK**	1:N	0..1 → 0..*	Partial / Partial	is assigned	`TASK.assignee_id`	A task may exist without an assignee
44	**USER**	**TASK**	1:N	1 → 0..*	Partial / Total	creates	`TASK.creator_id`	
45	**PROJECT**	**CONVERSATION**	1:1	0..1 → 0..1	Partial / Partial	has team chat	`CONVERSATION.project_id`	One team conversation per project
46	**CONVERSATION**	**CONVERSATION_PARTICIPANT**	1:N	1 → 1..*	Total / Total	includes	`CONVERSATION_PARTICIPANT.conversation_id`	
47	**USER**	**CONVERSATION_PARTICIPANT**	1:N	1 → 0..*	Partial / Total	takes part in	`CONVERSATION_PARTICIPANT.user_id`	
48	**CONVERSATION**	**MESSAGE**	1:N	1 → 0..*	Partial / Total	contains	`MESSAGE.conversation_id`	
49	**USER**	**MESSAGE**	1:N	1 → 0..*	Partial / Total	sends	`MESSAGE.sender_id`	
50	**USER**	**NOTIFICATION**	1:N	1 → 0..*	Partial / Total	is notified by	`NOTIFICATION.user_id`	Recipient
51	**PROJECT**	**NOTIFICATION**	1:N	0..1 → 0..*	Partial / Partial	triggers	`NOTIFICATION.project_id`	
52	**USER**	**AI_USAGE_COUNTER**	1:N	1 → 0..*	Partial / Total	consumes	`AI_USAGE_COUNTER.user_id`	Daily cap on AI requests

4. Many-to-Many Resolution

Conceptual M:N	Associative entity	Extra attributes	Used for
USER ↔ SKILL	**USER_SKILL**	`proficiency, source, added_at`	Profile (confirmed skills)
PROJECT ↔ SKILL	**PROJECT_SKILL**	`is_required`	Tech stack and required skills
PROJECT_ROLE ↔ SKILL	**ROLE_SKILL**	–	Skills of each role
USER ↔ PROJECT (membership)	**PROJECT_MEMBER**	`role_id, member_type, status, joined_at, left_at`	Team
USER ↔ PROJECT (save)	**SAVED_PROJECT**	`saved_at`	Saved projects
USER ↔ PROJECT (interest)	**INTEREST**	`created_at`	Express Interest
USER ↔ PROJECT (apply)	**JOIN_REQUEST**	`pitch_message, status, role_id, ...`	Request to Join (once only)
MATCH ↔ SKILL	**MATCH_SKILL**	`match_type, learning_tip`	Skill gap and matched skills
USER ↔ CONVERSATION	**CONVERSATION_PARTICIPANT**	`joined_at, last_read_at`	Chat
USER ↔ DETECTED_SKILL ↔ REPO	**SKILL_EVIDENCE**	`evidence_type, summary, ...`	Evidence for an inferred skill

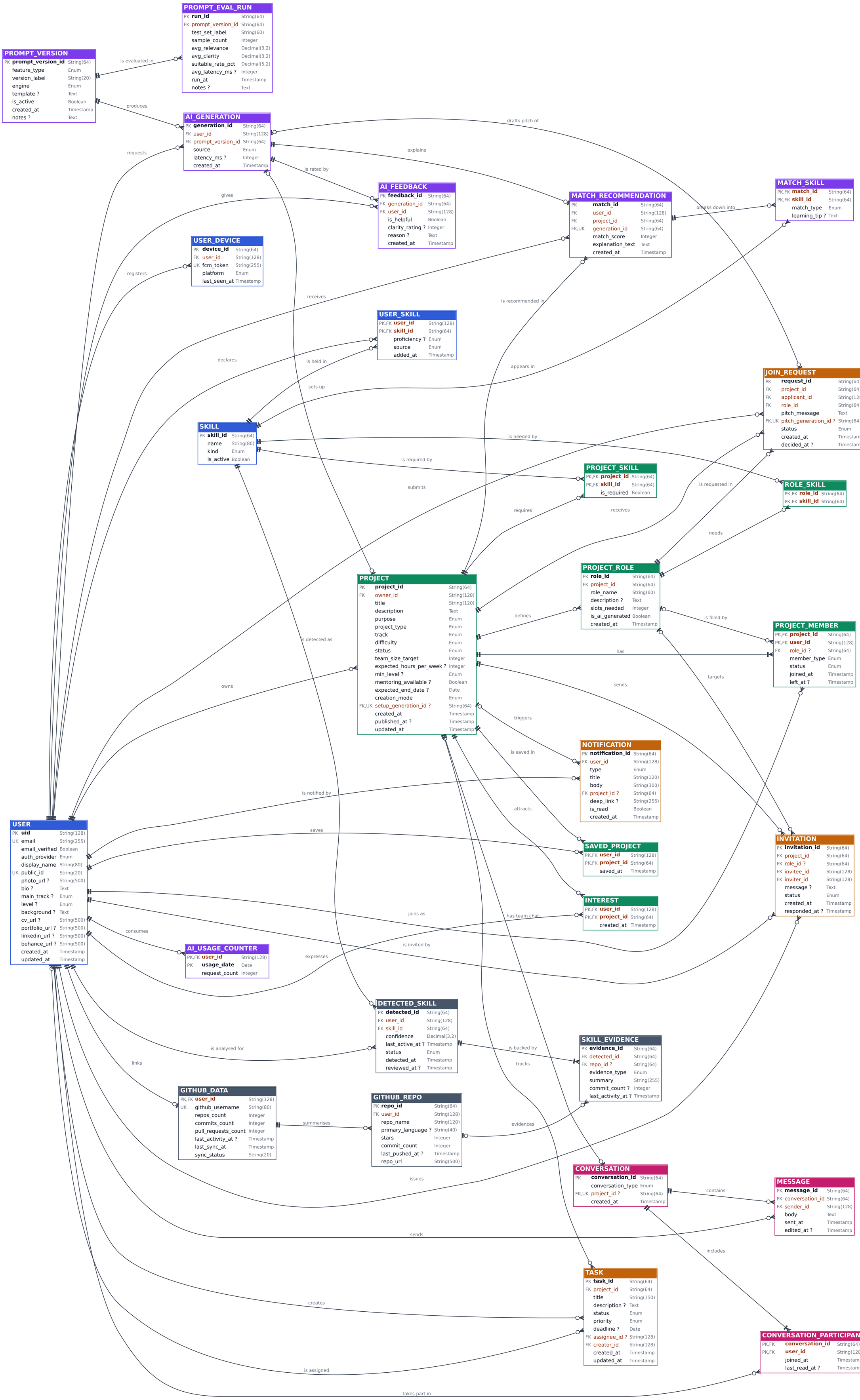
5. Overview ERD — all entities and relationships

The next page shows the map of all entities (names only) and the relationships between them. The full diagram with every attribute is in section 6.


PROMPT_VERSION
is evaluated in
PROMPT_EVAL_RUN
produces
AI_GENERATION
drafts pitch of
requests
explains
is rated by
gives
AI_FEEDBACK
MATCH_RECOMMENDATION
breaks down into
MATCH_SKILL
registers
USER_DEVICE
receives
is recommended in
USER_SKILL
is held in
appears in
declares
sets up
SKILL
JOIN_REQUEST
is needed by
is required by
PROJECT_SKILL
submits
is requested in
ROLE_SKILL
requires
receives
needs
PROJECT_ROLE
defines
is filled by
PROJECT_MEMBER
is detected as
has
owns
PROJECT
targets
sends
triggers
is saved in
NOTIFICATION
is notified by
attracts
SAVED_PROJECT
saves
INTEREST
USER
joins as
has team chat
INVITATION
is invited by
consumes
AI_USAGE_COUNTER
expresses
DETECTED_SKILL
is backed by
SKILL_EVIDENCE
is analysed for
tracks
links
GITHUB_DATA
summarises
GITHUB_REPO
evidences
issues
CONVERSATION
contains
MESSAGE
sends
includes
creates
TASK
is assigned
CONVERSATION_PARTICIPANT
takes part in

6. Detailed ERD — whole application (all attributes)

The complete diagram with every attribute. The page is intentionally large (vector), so zoom in as needed.


PROMPT_EVAL_RUN
PK run_id String(64)
FK prompt_version_id String(64)
test_set_label String(60)
sample_count Integer
avg_relevance Decimal(3,2)
avg_clarity Decimal(3,2)
suitable_rate_pct Decimal(5,2)
avg_latency_ms ? Integer
run_at Timestamp
notes ? Text
PROMPT_VERSION
PK prompt_version_id String(64)
feature_type Enum
version_label String(20)
engine Enum
template ? Text
is_active Boolean
created_at Timestamp
notes ? Text
is evaluated in
produces
AI_GENERATION
PK generation_id String(64)
FK user_id String(128)
FK prompt_version_id String(64)
source Enum
latency_ms ? Integer
created_at Timestamp
drafts pitch of
explains
requests
is rated by
gives
AI_FEEDBACK
PK feedback_id String(64)
FK generation_id String(64)
FK user_id String(128)
is_helpful Boolean
clarity_rating ? Integer
reason ? Text
created_at Timestamp
MATCH_RECOMMENDATION
PK match_id String(64)
FK user_id String(128)
FK project_id String(64)
FK,UK generation_id String(64)
match_score Integer
explanation_text Text
created_at Timestamp
breaks down into
MATCH_SKILL
PK,FK match_id String(64)
PK,FK skill_id String(64)
match_type Enum
learning_tip ? Text
USER_DEVICE
PK device_id String(64)
FK user_id String(128)
UK fcm_token String(255)
platform Enum
last_seen_at Timestamp
registers
receives
is recommended in
USER_SKILL
PK,FK user_id String(128)
PK,FK skill_id String(64)
proficiency ? Enum
source Enum
added_at Timestamp
declares
is held in
appears in
sets up
JOIN_REQUEST
PK request_id String(64)
FK project_id String(64)
FK applicant_id String(128)
FK role_id String(64)
pitch_message Text
FK,UK pitch_generation_id ? String(64)
status Enum
created_at Timestamp
decided_at ? Timestamp
SKILL
PK skill_id String(64)
name String(80)
kind Enum
is_active Boolean
is needed by
is required by
PROJECT_SKILL
PK,FK project_id String(64)
PK,FK skill_id String(64)
is_required Boolean
ROLE_SKILL
PK,FK role_id String(64)
PK,FK skill_id String(64)
is requested in
submits
requires
receives
needs
PROJECT
PK project_id String(64)
FK owner_id String(128)
title String(120)
description Text
purpose Enum
project_type Enum
track Enum
difficulty Enum
status Enum
team_size_target Integer
expected_hours_per_week ? Integer
min_level ? Enum
mentoring_available ? Boolean
expected_end_date ? Date
creation_mode Enum
FK,UK setup_generation_id ? String(64)
created_at Timestamp
published_at ? Timestamp
updated_at Timestamp
PROJECT_ROLE
PK role_id String(64)
FK project_id String(64)
role_name String(60)
description ? Text
slots_needed Integer
is_ai_generated Boolean
created_at Timestamp
defines
is filled by
PROJECT_MEMBER
PK,FK project_id String(64)
PK,FK user_id String(128)
FK role_id ? String(64)
member_type Enum
status Enum
joined_at Timestamp
left_at ? Timestamp
is detected as
has
owns
sends
targets
triggers
NOTIFICATION
PK notification_id String(64)
FK user_id String(128)
type Enum
title String(120)
body String(300)
FK project_id ? String(64)
deep_link ? String(255)
is_read Boolean
created_at Timestamp
is saved in
is notified by
attracts
saves
SAVED_PROJECT
PK,FK user_id String(128)
PK,FK project_id String(64)
saved_at Timestamp
INTEREST
PK,FK user_id String(128)
PK,FK project_id String(64)
created_at Timestamp
has team chat
INVITATION
PK invitation_id String(64)
FK project_id String(64)
FK role_id ? String(64)
FK invitee_id String(128)
FK inviter_id String(128)
message ? Text
status Enum
created_at Timestamp
responded_at ? Timestamp
USER
PK uid String(128)
UK email String(255)
email_verified Boolean
auth_provider Enum
display_name String(80)
UK public_id String(20)
photo_url ? String(500)
bio ? Text
main_track ? Enum
level ? Enum
background ? Text
cv_url ? String(500)
portfolio_url ? String(500)
linkedin_url ? String(500)
behance_url ? String(500)
created_at Timestamp
updated_at Timestamp
joins as
is invited by
consumes
AI_USAGE_COUNTER
PK,FK user_id String(128)
PK usage_date Date
request_count Integer
expresses
DETECTED_SKILL
PK detected_id String(64)
FK user_id String(128)
FK skill_id String(64)
confidence Decimal(3,2)
last_active_at ? Timestamp
status Enum
detected_at Timestamp
reviewed_at ? Timestamp
is analysed for
is backed by
tracks
links
SKILL_EVIDENCE
PK evidence_id String(64)
FK detected_id String(64)
FK repo_id ? String(64)
evidence_type Enum
summary String(255)
commit_count ? Integer
last_activity_at ? Timestamp
GITHUB_DATA
PK,FK user_id String(128)
UK github_username String(80)
repos_count Integer
commits_count Integer
pull_requests_count Integer
last_activity_at ? Timestamp
last_sync_at Timestamp
sync_status String(20)
GITHUB_REPO
PK repo_id String(64)
FK user_id String(128)
repo_name String(120)
primary_language ? String(40)
stars Integer
commit_count Integer
last_pushed_at ? Timestamp
repo_url String(500)
summarises
evidences
issues
CONVERSATION
PK conversation_id String(64)
conversation_type Enum
FK,UK project_id ? String(64)
created_at Timestamp
contains
MESSAGE
PK message_id String(64)
FK conversation_id String(64)
FK sender_id String(128)
body Text
sent_at Timestamp
edited_at ? Timestamp
includes
creates
TASK
PK task_id String(64)
FK project_id String(64)
title String(150)
description ? Text
status Enum
priority Enum
deadline ? Date
FK assignee_id ? String(128)
FK creator_id String(128)
created_at Timestamp
updated_at Timestamp
is assigned
CONVERSATION_PARTICIPANT
PK,FK conversation_id String(64)
PK,FK user_id String(128)
joined_at Timestamp
last_read_at ? Timestamp
takes part in

7. Feature ERDs (11 Core Features in scope)

One section for each Core Feature that remains in scope: the purpose of its data model, its entities, a specialised ERD, its relationships and the business rules taken from the documents. **Note:** features 7 (Community) and 13 (Chatbot) of the Core Features document are excluded from this project scope; the original feature numbers are kept for traceability.

#	Feature	Entities	Core entities
1	**Splash & Onboarding**	0	— (no persisted entities)
2	**Authentication (with Guest Mode)**	2	USER, USER_DEVICE
3	**Profile (with Profile Completion)**	3	USER, SKILL, USER_SKILL
4	**GitHub Integration**	4	GITHUB_DATA, GITHUB_REPO, DETECTED_SKILL, SKILL_EVIDENCE
5	**AI Match**	5	MATCH_RECOMMENDATION, MATCH_SKILL, AI_GENERATION, AI_FEEDBACK, AI_USAGE_COUNTER
6	**Project Discovery**	5	PROJECT, PROJECT_SKILL, PROJECT_ROLE, SAVED_PROJECT, INTEREST
8	**Project Details**	6	PROJECT, PROJECT_SKILL, PROJECT_ROLE, JOIN_REQUEST, SAVED_PROJECT, INTEREST
9	**Create Project**	5	PROJECT, PROJECT_SKILL, PROJECT_ROLE, ROLE_SKILL, PROJECT_MEMBER
10	**My Teams**	6	PROJECT_MEMBER, TASK, CONVERSATION, CONVERSATION_PARTICIPANT, MESSAGE, NOTIFICATION
11	**My Projects**	4	PROJECT_MEMBER, JOIN_REQUEST, INVITATION, NOTIFICATION
12	**Prompt Evaluation**	4	PROMPT_VERSION, PROMPT_EVAL_RUN, AI_GENERATION, AI_FEEDBACK

Feature 1 — Splash & Onboarding

No data is persisted in the database. The first-launch flag lives locally in SharedPreferences, and the existing-session check reads Firebase Auth state.

Business rules from the documents

- The onboarding flag is local (SharedPreferences) - not part of the schema.
- The session check only reads the Firebase Auth state.

Feature 2 — Authentication (with Guest Mode)

Sign-in uses Firebase Auth (Email + Google). A Guest is not an entity: there is no account and no data. USER_DEVICE stores the FCM token after sign-in.

Entities

Entity	Module	Role in this feature
USER	Identity & Profile	A developer / project owner account: account data and the public profile. A Guest is not stored here (no account, nothing persisted).
USER_DEVICE	Identity & Profile	User devices and their FCM token, used to deliver push notifications (one or more devices per user).

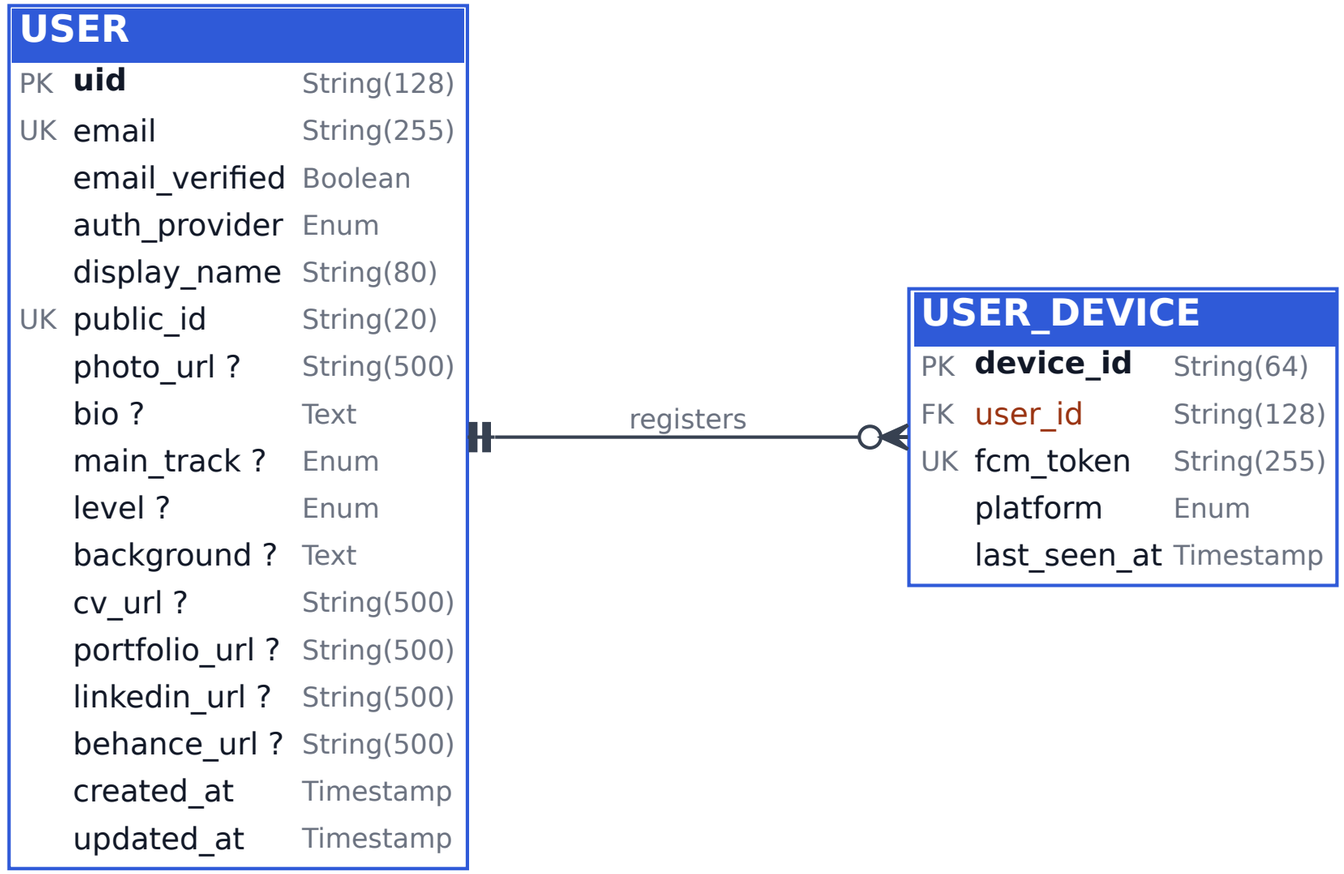
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	registers	**USER_DEVICE**	1:N (1 → 0..*)	`USER_DEVICE.user_id`

Business rules from the documents

- Guest = unauthenticated, read-only user; no row in the database.
- Delete account (from Settings) removes USER and all dependent data (cascade via a Cloud Function).
- GitHub is connected from inside the Profile (not a sign-in method); LinkedIn is only a link in the profile (FR-01).

ERD — Feature 2: Authentication (with Guest Mode)



Feature 3 — Profile (with Profile Completion)

Core developer data. Confirmed skills are modelled with the associative table USER_SKILL, which resolves the M:N relationship with the SKILL catalogue. The completion indicator is computed, not stored, in the logical schema.

Entities

Entity	Module	Role in this feature
USER	Identity & Profile	A developer / project owner account: account data and the public profile. A Guest is not stored here (no account, nothing persisted).
SKILL	Identity & Profile	Skill catalogue (programming languages, technical skills, soft skills, interests) shared by profiles and projects.
USER_SKILL	Identity & Profile	Associative table (M:N) between users and their confirmed skills. Each programming language carries a level.

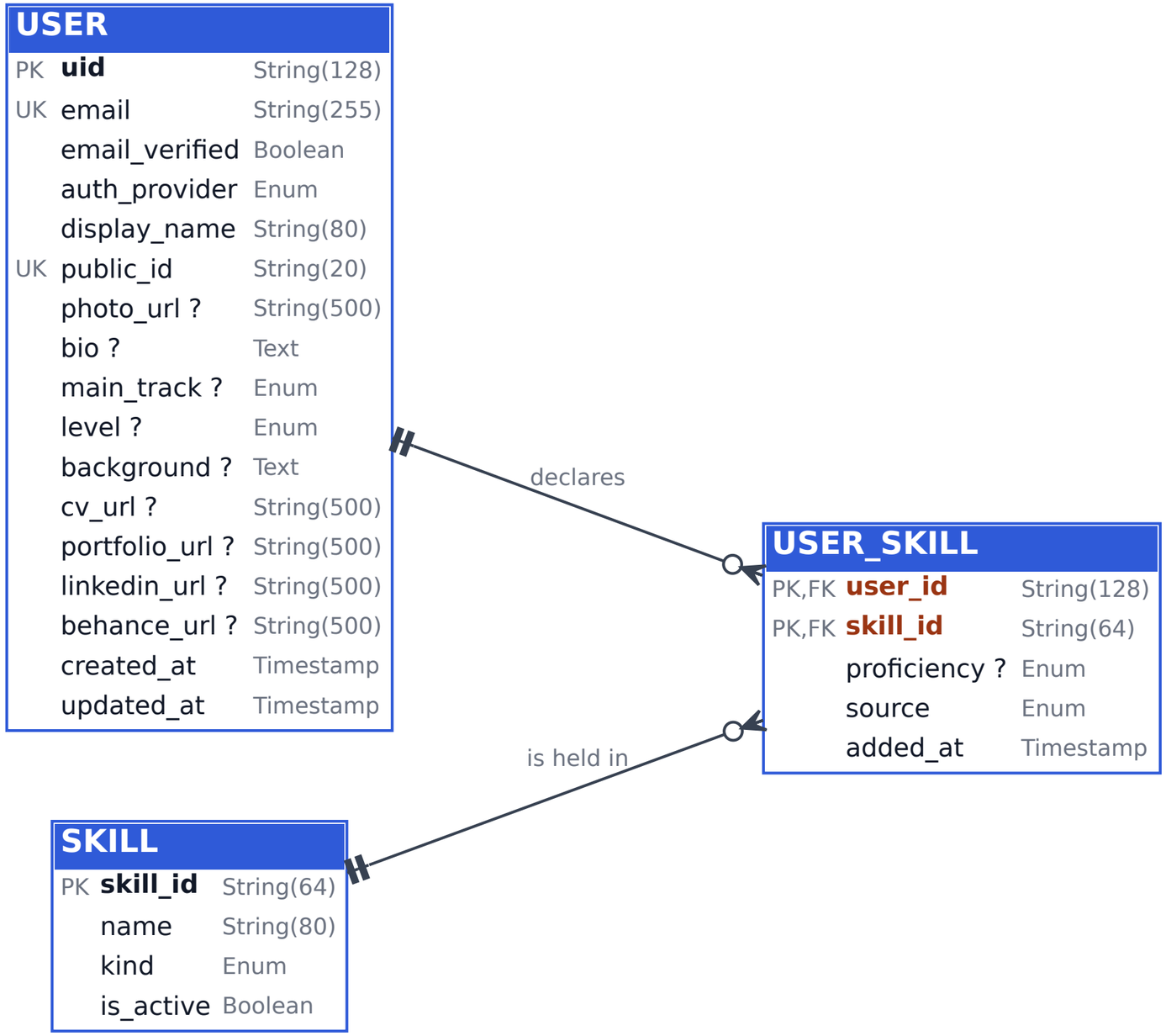
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	declares	**USER_SKILL**	1:N (1 → 0..*)	`USER_SKILL.user_id`
SKILL	is held in	**USER_SKILL**	1:N (1 → 0..*)	`USER_SKILL.skill_id`

Business rules from the documents

- Mandatory fields: name, Main track, Level, at least one programming language (US-02).
- The user ID (public_id) is unique and auto-generated.
- The CV is stored in Firebase Storage; Firestore keeps only its path.
- Profile Completion is a computed indicator (target: 80% of testers complete it).

ERD — Feature 3: Profile (with Profile Completion)



Feature 4 — GitHub Integration

Inferred skills stay in DETECTED_SKILL with status pending until the user approves them, then they are copied to USER_SKILL. Each inferred skill has evidence (SKILL_EVIDENCE).

Entities

Entity	Module	Role in this feature
GITHUB_DATA	GitHub Integration	GitHub connection summary for a user (1:1). The OAuth token is NOT stored here (NFR-07).
GITHUB_REPO	GitHub Integration	Repository summaries pulled from GitHub (top languages are computed from them).
DETECTED_SKILL	GitHub Integration	A skill inferred by the AI from GitHub. It is NOT a verified fact until the user accepts it (status).
SKILL_EVIDENCE	GitHub Integration	Evidence backing an inferred skill (repo / commits / PR). Resolves the multi-valued attribute.

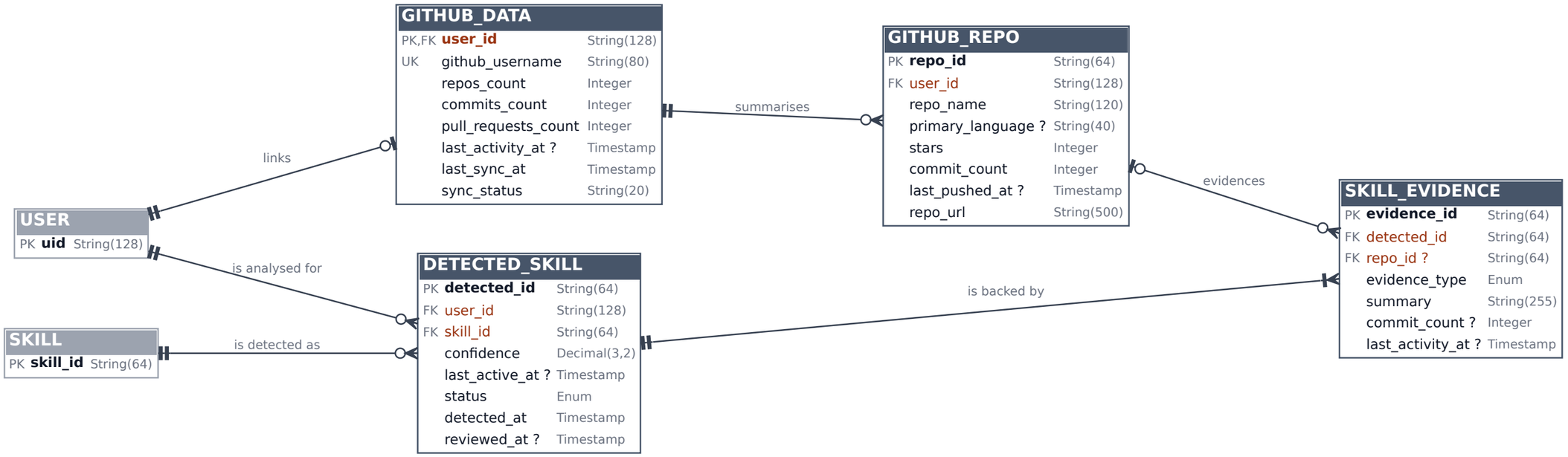
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	links	**GITHUB_DATA**	1:1 (1 → 0..1)	`GITHUB_DATA.user_id`
GITHUB_DATA	summarises	**GITHUB_REPO**	1:N (1 → 0..*)	`GITHUB_REPO.user_id`
USER	is analysed for	**DETECTED_SKILL**	1:N (1 → 0..*)	`DETECTED_SKILL.user_id`
SKILL	is detected as	**DETECTED_SKILL**	1:N (1 → 0..*)	`DETECTED_SKILL.skill_id`
DETECTED_SKILL	is backed by	**SKILL_EVIDENCE**	1:N (1 → 1..*)	`SKILL_EVIDENCE.detected_id`
GITHUB_REPO	evidences	**SKILL_EVIDENCE**	1:N (0..1 → 0..*)	`SKILL_EVIDENCE.repo_id`

Business rules from the documents

- An inferred skill is not a verified fact: it is shown in a separate section (AI-Detected Skills) and the user accepts or rejects it.
- The GitHub activity score is not a judgement on the person (no "trusted / untrusted" label).
- The token is not stored in Firestore (secure storage or server-side - NFR-07).
- Disconnect removes githubData and detectedSkills (after confirmation).

ERD — Feature 4: GitHub Integration



Feature 5 — AI Match

Every recommendation has an explanation plus matched / complementary / missing skills with a learning tip. It is saved in Match History and tied to the prompt version that produced it. AI_USAGE_COUNTER caps daily AI requests per user (Risk R5).

Entities

Entity	Module	Role in this feature
MATCH_RECOMMENDATION	AI, Matching & Evaluation	A project recommendation for a user plus its explanation. All rows together form the Match History.
MATCH_SKILL	AI, Matching & Evaluation	Per-recommendation detail: matched / complementary / close / missing skills, with a short learning tip for missing ones.
AI_GENERATION	AI, Matching & Evaluation	Log of every AI output (recommendation, project setup, pitch). Links an output to its prompt version.
AI_FEEDBACK	AI, Matching & Evaluation	User rating of an AI output: thumbs up/down + clarity 1-5 + optional reason.
AI_USAGE_COUNTER	AI, Matching & Evaluation	Daily counter of AI requests per user (cost control).

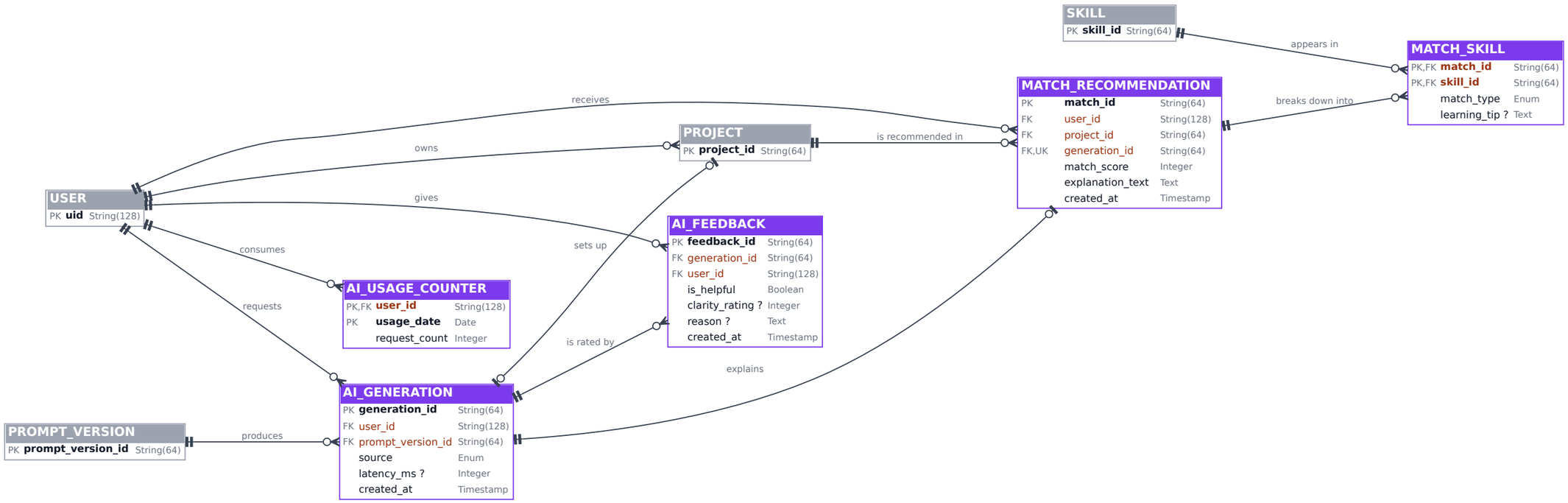
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
AI_GENERATION	sets up	**PROJECT**	1:1 (0..1 → 0..1)	`PROJECT.setup_generation_id`
PROMPT_VERSION	produces	**AI_GENERATION**	1:N (1 → 0..*)	`AI_GENERATION.prompt_version_id`
USER	requests	**AI_GENERATION**	1:N (1 → 0..*)	`AI_GENERATION.user_id`
AI_GENERATION	explains	**MATCH_RECOMMENDATION**	1:1 (1 → 0..1)	`MATCH_RECOMMENDATION.generation_id`
USER	receives	**MATCH_RECOMMENDATION**	1:N (1 → 0..*)	`MATCH_RECOMMENDATION.user_id`
PROJECT	is recommended in	**MATCH_RECOMMENDATION**	1:N (1 → 0..*)	`MATCH_RECOMMENDATION.project_id`
MATCH_RECOMMENDATION	breaks down into	**MATCH_SKILL**	1:N (1 → 0..*)	`MATCH_SKILL.match_id`
SKILL	appears in	**MATCH_SKILL**	1:N (1 → 0..*)	`MATCH_SKILL.skill_id`
AI_GENERATION	is rated by	**AI_FEEDBACK**	1:N (1 → 0..*)	`AI_FEEDBACK.generation_id`
USER	gives	**AI_FEEDBACK**	1:N (1 → 0..*)	`AI_FEEDBACK.user_id`
USER	consumes	**AI_USAGE_COUNTER**	1:N (1 → 0..*)	`AI_USAGE_COUNTER.user_id`

Business rules from the documents

- No bare score: explanation_text is mandatory (Explainable matching).
- If the LLM is slow: Fallback (cached / rule-based) is recorded with source = Fallback (NFR-11).
- Thumbs up/down + clarity 1-5 are stored in AI_FEEDBACK.
- If the profile is incomplete: no recommendation; the user sees "Complete your profile" (UC-02 alternative path).

ERD — Feature 5: AI Match



Feature 6 — Project Discovery

Discovery reads PROJECT and its child tables. Save and Express Interest are separate entities. Filters rely on columns of PROJECT, PROJECT_SKILL and PROJECT_ROLE.

Entities

Entity	Module	Role in this feature
PROJECT	Projects & Teams	A project (created manually or AI-assisted). Project Purpose is mandatory and drives related fields.
PROJECT_SKILL	Projects & Teams	Associative table (M:N): the project's tech stack and required skills.
PROJECT_ROLE	Projects & Teams	Roles a project needs (Frontend, Backend, QA ...). AI-generated and editable.
SAVED_PROJECT	Projects & Teams	Saved projects. Save and Favorites are merged into a single Saved list.
INTEREST	Projects & Teams	Express Interest: an action separate from Save and Request to Join. At most once per user and project.

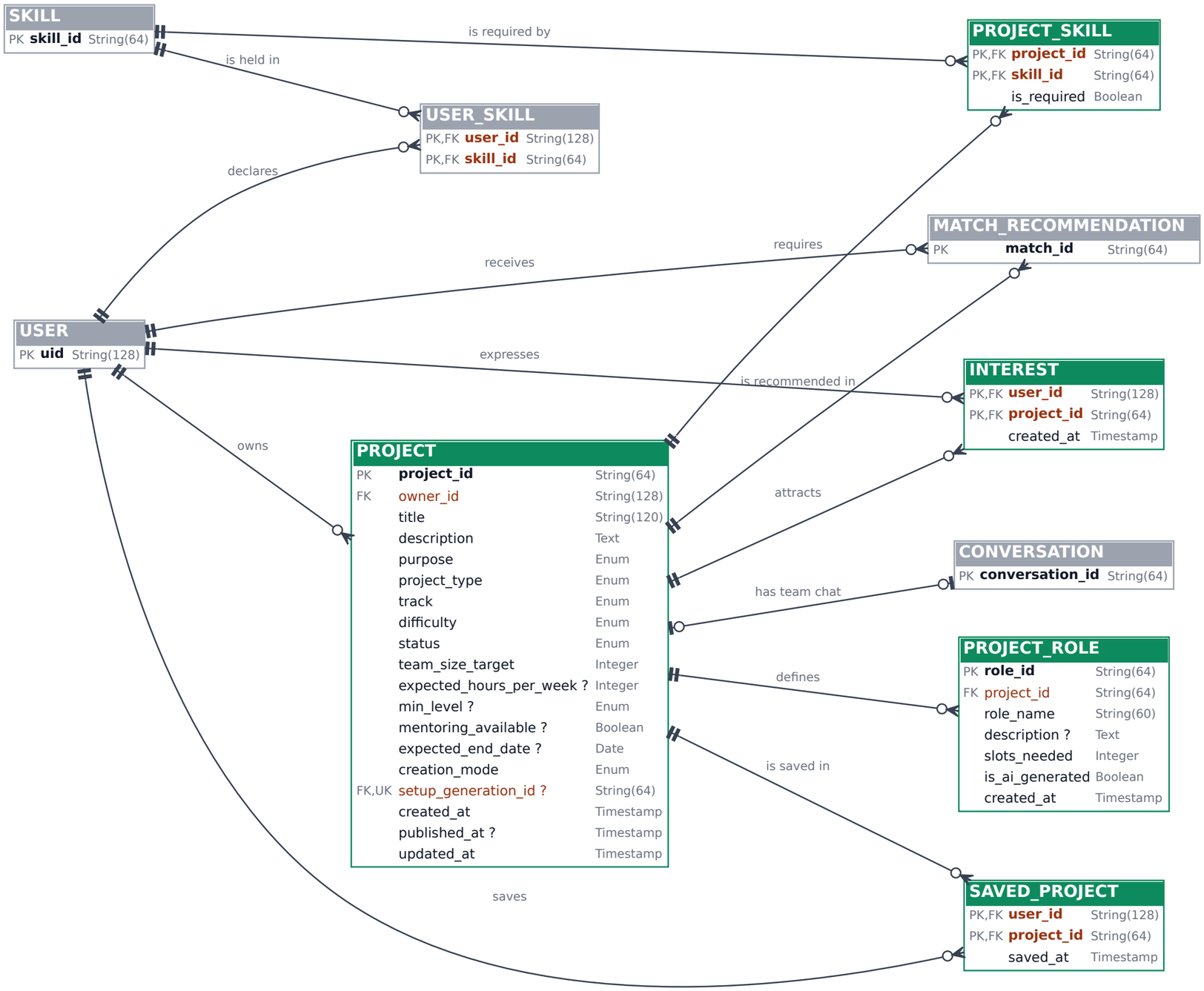
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	owns	**PROJECT**	1:N (1 → 0..*)	`PROJECT.owner_id`
PROJECT	requires	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.project_id`
SKILL	is required by	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.skill_id`
PROJECT	defines	**PROJECT_ROLE**	1:N (1 → 0..*)	`PROJECT_ROLE.project_id`
USER	saves	**SAVED_PROJECT**	1:N (1 → 0..*)	`SAVED_PROJECT.user_id`
PROJECT	is saved in	**SAVED_PROJECT**	1:N (1 → 0..*)	`SAVED_PROJECT.project_id`
USER	expresses	**INTEREST**	1:N (1 → 0..*)	`INTEREST.user_id`
PROJECT	attracts	**INTEREST**	1:N (1 → 0..*)	`INTEREST.project_id`
PROJECT	is recommended in	**MATCH_RECOMMENDATION**	1:N (1 → 0..*)	`MATCH_RECOMMENDATION.project_id`
PROJECT	has team chat	**CONVERSATION**	1:1 (0..1 → 0..1)	`CONVERSATION.project_id`

Business rules from the documents

- Filters: Tech stack, Difficulty, Project purpose, Project type, Roles needed, Team size, Track, Status.
- Save and Favorites are merged into Saved; Save, Express Interest and Request to Join are three separate actions.
- A Guest is read-only (Discover, Project Details, Public Profiles).
- Start chat / Invite to project from a developer profile requires login.

ERD — Feature 6: Project Discovery



Feature 8 — Project Details

The decision page: project data + Project Purpose badge + match + skill gap. Request to Join is stored in JOIN_REQUEST (once only) together with the AI pitch.

Entities

Entity	Module	Role in this feature
PROJECT	Projects & Teams	A project (created manually or AI-assisted). Project Purpose is mandatory and drives related fields.
PROJECT_SKILL	Projects & Teams	Associative table (M:N): the project's tech stack and required skills.
PROJECT_ROLE	Projects & Teams	Roles a project needs (Frontend, Backend, QA ...). AI-generated and editable.
JOIN_REQUEST	Collaboration & Workspace	A request to join a project. Sent once only (UNIQUE) with status Pending / Accepted / Rejected.
SAVED_PROJECT	Projects & Teams	Saved projects. Save and Favorites are merged into a single Saved list.
INTEREST	Projects & Teams	Express Interest: an action separate from Save and Request to Join. At most once per user and project.

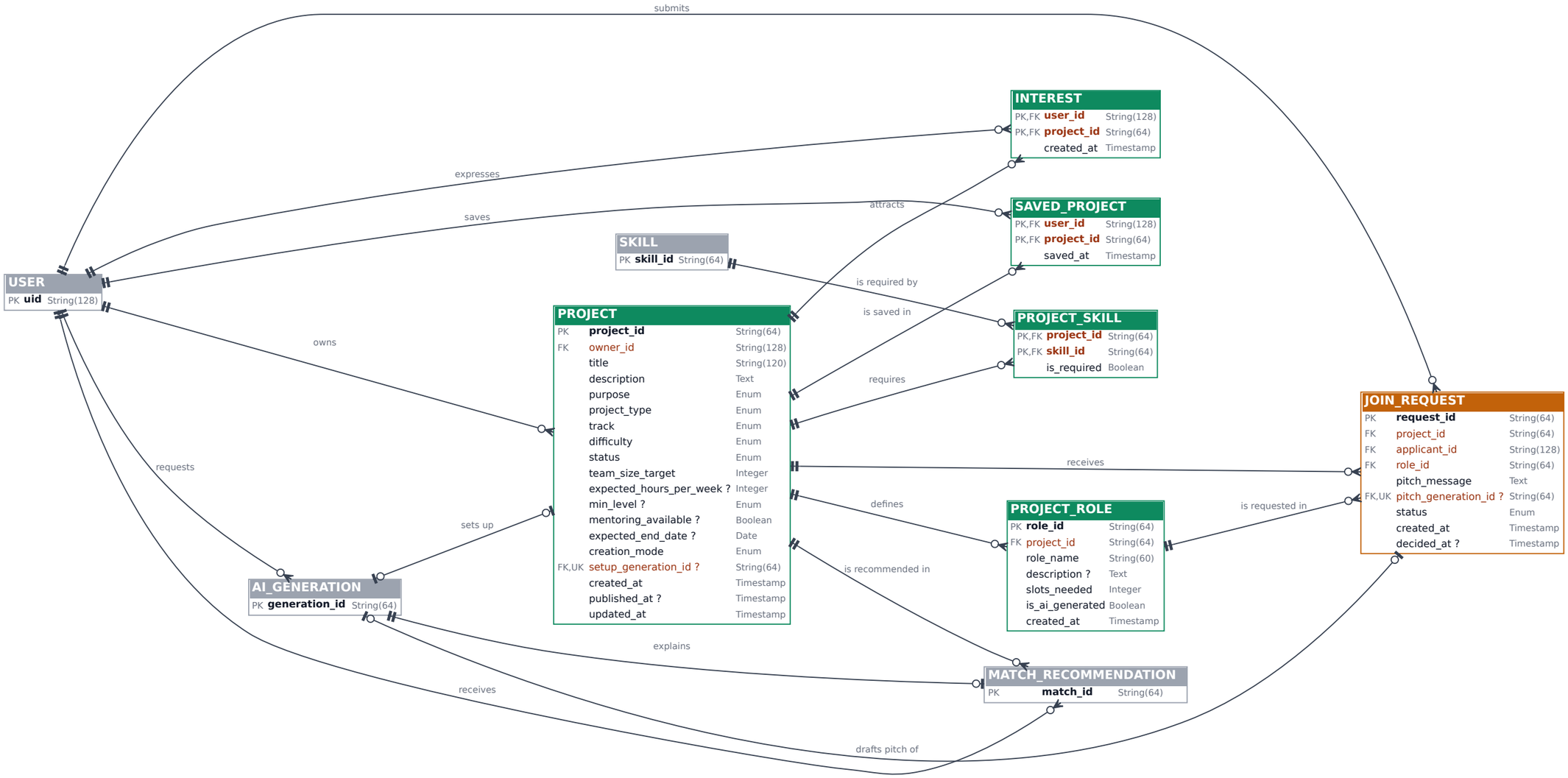
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	owns	**PROJECT**	1:N (1 → 0..*)	`PROJECT.owner_id`
PROJECT	requires	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.project_id`
SKILL	is required by	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.skill_id`
PROJECT	defines	**PROJECT_ROLE**	1:N (1 → 0..*)	`PROJECT_ROLE.project_id`
USER	saves	**SAVED_PROJECT**	1:N (1 → 0..*)	`SAVED_PROJECT.user_id`
PROJECT	is saved in	**SAVED_PROJECT**	1:N (1 → 0..*)	`SAVED_PROJECT.project_id`
USER	expresses	**INTEREST**	1:N (1 → 0..*)	`INTEREST.user_id`
PROJECT	attracts	**INTEREST**	1:N (1 → 0..*)	`INTEREST.project_id`
PROJECT	receives	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.project_id`
USER	submits	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.applicant_id`
PROJECT_ROLE	is requested in	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.role_id`
AI_GENERATION	drafts pitch of	**JOIN_REQUEST**	1:1 (0..1 → 0..1)	`JOIN_REQUEST.pitch_generation_id`
AI_GENERATION	sets up	**PROJECT**	1:1 (0..1 → 0..1)	`PROJECT.setup_generation_id`
PROJECT	is recommended in	**MATCH_RECOMMENDATION**	1:N (1 → 0..*)	`MATCH_RECOMMENDATION.project_id`

Business rules from the documents

- Project Purpose is mandatory: Learning (beginner-friendly, optional mentoring, no strict deadline) / Real_World (hours per week, minimum level, deadline).
- AI Match takes the purpose into account (a beginner is shown Learning projects first).
- A request is sent once and duplicates are blocked (UNIQUE project_id + applicant_id).
- Request status: Pending / Accepted / Rejected.

ERD — Feature 8: Project Details



Feature 9 — Create Project

Projects are created manually or with AI. A draft is saved with status Draft and publishing needs the owner's approval. The owner is automatically recorded in PROJECT_MEMBER as Owner.

Entities

Entity	Module	Role in this feature
PROJECT	Projects & Teams	A project (created manually or AI-assisted). Project Purpose is mandatory and drives related fields.
PROJECT_SKILL	Projects & Teams	Associative table (M:N): the project's tech stack and required skills.
PROJECT_ROLE	Projects & Teams	Roles a project needs (Frontend, Backend, QA ...). AI-generated and editable.
ROLE_SKILL	Projects & Teams	Associative table: skills required by each role (supports Missing Role Alert and Skill Gap). Design addition.
PROJECT_MEMBER	Projects & Teams	Team membership (associative between user and project). The owner is recorded as an Owner member.

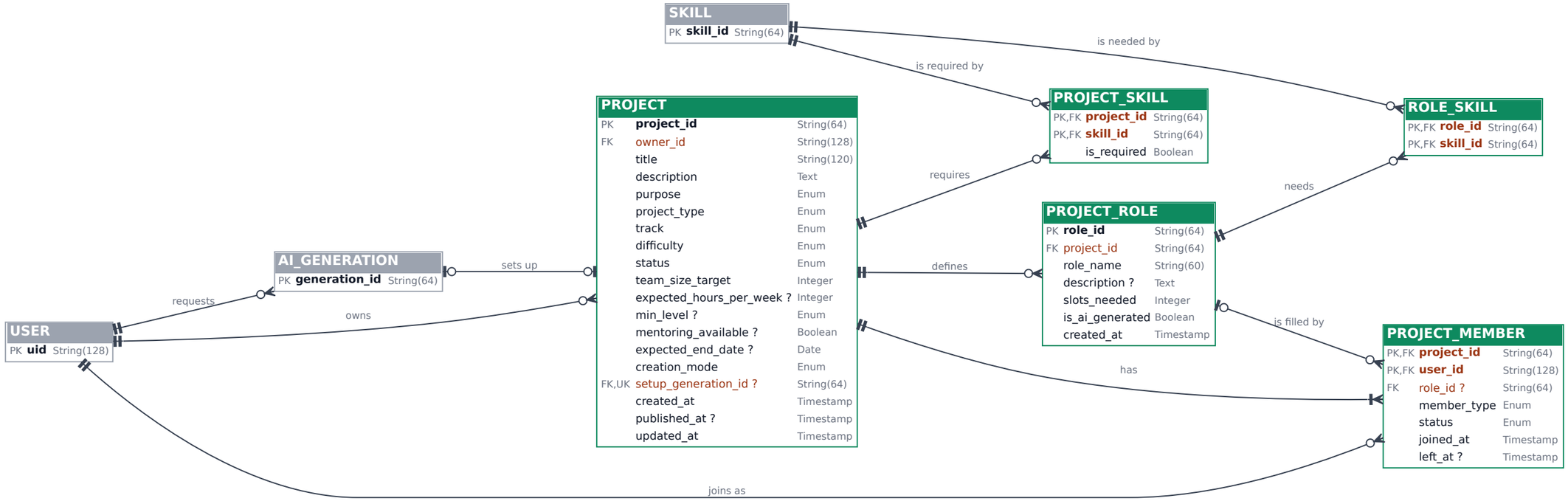
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
USER	owns	**PROJECT**	1:N (1 → 0..*)	`PROJECT.owner_id`
PROJECT	requires	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.project_id`
SKILL	is required by	**PROJECT_SKILL**	1:N (1 → 0..*)	`PROJECT_SKILL.skill_id`
PROJECT	defines	**PROJECT_ROLE**	1:N (1 → 0..*)	`PROJECT_ROLE.project_id`
PROJECT_ROLE	needs	**ROLE_SKILL**	1:N (1 → 0..*)	`ROLE_SKILL.role_id`
SKILL	is needed by	**ROLE_SKILL**	1:N (1 → 0..*)	`ROLE_SKILL.skill_id`
PROJECT	has	**PROJECT_MEMBER**	1:N (1 → 1..*)	`PROJECT_MEMBER.project_id`
USER	joins as	**PROJECT_MEMBER**	1:N (1 → 0..*)	`PROJECT_MEMBER.user_id`
PROJECT_ROLE	is filled by	**PROJECT_MEMBER**	1:N (0..1 → 0..*)	`PROJECT_MEMBER.role_id`
AI_GENERATION	sets up	**PROJECT**	1:1 (0..1 → 0..1)	`PROJECT.setup_generation_id`

Business rules from the documents

- All AI-generated content is editable before publishing; there is no auto-publish.
- If the AI fails: manual entry with a clear message.
- Review and Publish shows the type summary (Learning / Real).
- Missing Role Alert: any role whose slots_needed exceeds the current members filling it.

ERD — Feature 9: Create Project



Feature 10 — My Teams

Daily workspace: members (PROJECT_MEMBER), team tasks (TASK), chat (CONVERSATION / MESSAGE) and assignment / update notifications.

Entities

Entity	Module	Role in this feature
PROJECT_MEMBER	Projects & Teams	Team membership (associative between user and project). The owner is recorded as an Owner member.
TASK	Collaboration & Workspace	A task on the board: To Do / In Progress / In Review / Done.
CONVERSATION	Messaging	A direct (1-to-1) conversation or a team conversation tied to a project.
CONVERSATION_PARTICIPANT	Messaging	Associative table (M:N) between conversations and users, plus last-read time.
MESSAGE	Messaging	A message inside a conversation (real-time).
NOTIFICATION	Collaboration & Workspace	An in-app notification (Today / Earlier list) with a reference for FCM and the deep link.

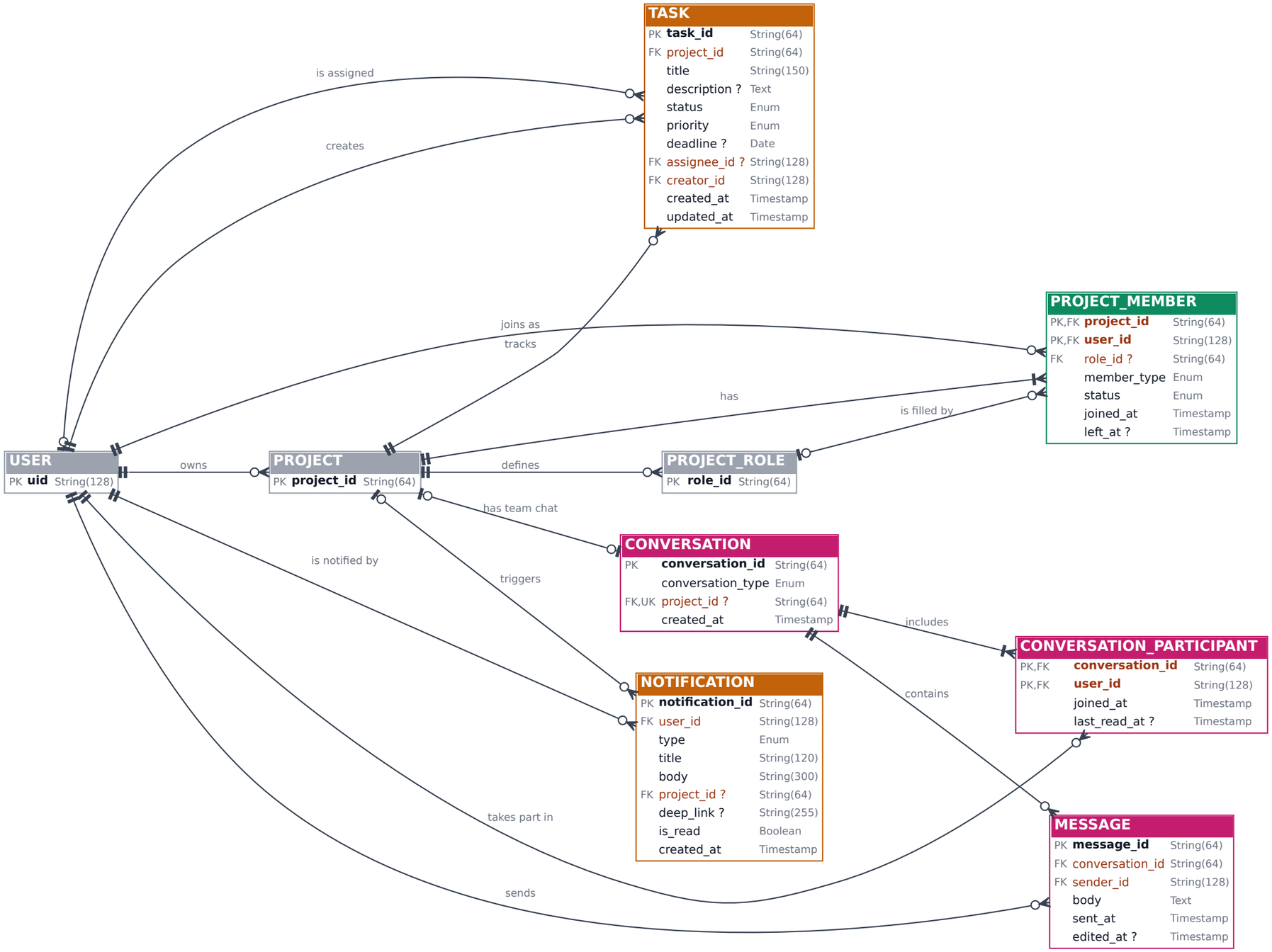
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
PROJECT	has	**PROJECT_MEMBER**	1:N (1 → 1..*)	`PROJECT_MEMBER.project_id`
USER	joins as	**PROJECT_MEMBER**	1:N (1 → 0..*)	`PROJECT_MEMBER.user_id`
PROJECT_ROLE	is filled by	**PROJECT_MEMBER**	1:N (0..1 → 0..*)	`PROJECT_MEMBER.role_id`
PROJECT	tracks	**TASK**	1:N (1 → 0..*)	`TASK.project_id`
USER	is assigned	**TASK**	1:N (0..1 → 0..*)	`TASK.assignee_id`
USER	creates	**TASK**	1:N (1 → 0..*)	`TASK.creator_id`
PROJECT	has team chat	**CONVERSATION**	1:1 (0..1 → 0..1)	`CONVERSATION.project_id`
CONVERSATION	includes	**CONVERSATION_PARTICIPANT**	1:N (1 → 1..*)	`CONVERSATION_PARTICIPANT.conversation_id`
USER	takes part in	**CONVERSATION_PARTICIPANT**	1:N (1 → 0..*)	`CONVERSATION_PARTICIPANT.user_id`
CONVERSATION	contains	**MESSAGE**	1:N (1 → 0..*)	`MESSAGE.conversation_id`
USER	sends	**MESSAGE**	1:N (1 → 0..*)	`MESSAGE.sender_id`
USER	is notified by	**NOTIFICATION**	1:N (1 → 0..*)	`NOTIFICATION.user_id`
PROJECT	triggers	**NOTIFICATION**	1:N (0..1 → 0..*)	`NOTIFICATION.project_id`

Business rules from the documents

- Task Board: To Do / In Progress / In Review / Done; each task has deadline, priority and assignee.
- Tasks = Must, Chat = Should.
- If the user is offline the operation is queued (UC-05).
- An FCM notification goes to the assignee when a task is assigned or updated.

ERD — Feature 10: My Teams



Feature 11 — My Projects

Manage projects by status and role (Owner / Member): incoming requests, invitations, and accept / reject with a notification within 10 seconds.

Entities

Entity	Module	Role in this feature
PROJECT_MEMBER	Projects & Teams	Team membership (associative between user and project). The owner is recorded as an Owner member.
JOIN_REQUEST	Collaboration & Workspace	A request to join a project. Sent once only (UNIQUE) with status Pending / Accepted / Rejected.
INVITATION	Collaboration & Workspace	An invitation sent to a developer (Invite Developer). The owner can cancel it.
NOTIFICATION	Collaboration & Workspace	An in-app notification (Today / Earlier list) with a reference for FCM and the deep link.

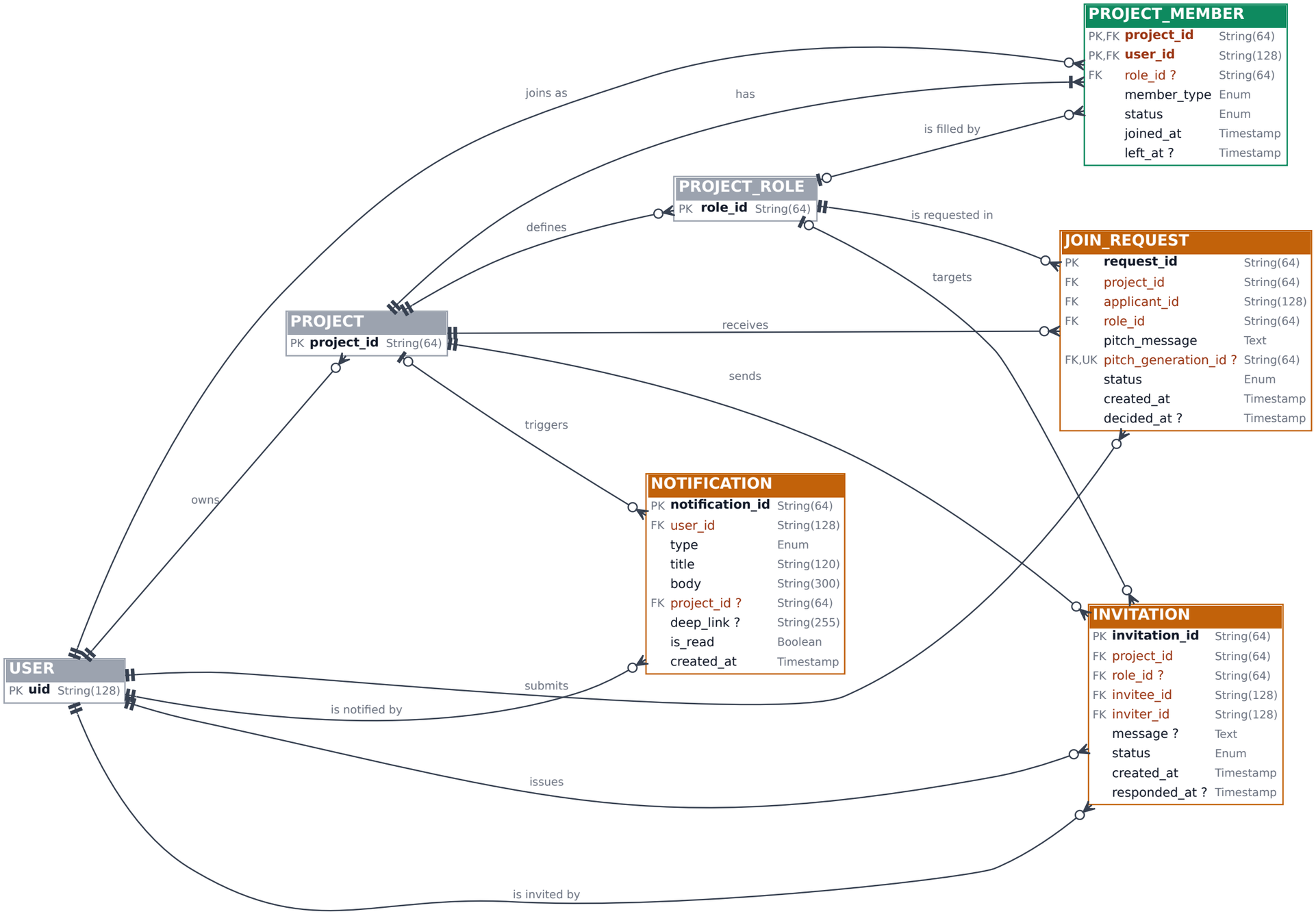
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
PROJECT	has	**PROJECT_MEMBER**	1:N (1 → 1..*)	`PROJECT_MEMBER.project_id`
USER	joins as	**PROJECT_MEMBER**	1:N (1 → 0..*)	`PROJECT_MEMBER.user_id`
PROJECT_ROLE	is filled by	**PROJECT_MEMBER**	1:N (0..1 → 0..*)	`PROJECT_MEMBER.role_id`
PROJECT	receives	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.project_id`
USER	submits	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.applicant_id`
PROJECT_ROLE	is requested in	**JOIN_REQUEST**	1:N (1 → 0..*)	`JOIN_REQUEST.role_id`
PROJECT	sends	**INVITATION**	1:N (1 → 0..*)	`INVITATION.project_id`
USER	is invited by	**INVITATION**	1:N (1 → 0..*)	`INVITATION.invitee_id`
USER	issues	**INVITATION**	1:N (1 → 0..*)	`INVITATION.inviter_id`
PROJECT_ROLE	targets	**INVITATION**	1:N (0..1 → 0..*)	`INVITATION.role_id`
USER	is notified by	**NOTIFICATION**	1:N (1 → 0..*)	`NOTIFICATION.user_id`
PROJECT	triggers	**NOTIFICATION**	1:N (0..1 → 0..*)	`NOTIFICATION.project_id`

Business rules from the documents

- Accepting a request adds the applicant to PROJECT_MEMBER and notifies both parties.
- Rejecting a request closes it and sends a notification.
- Project status changes: Open / Full / Completed (and Archive).
- An invitation can be cancelled by the owner (Cancelled).

ERD — Feature 11: My Projects



Feature 12 — Prompt Evaluation

Stores prompt versions (v1, v2 ...), ties every AI output to its version, aggregates ratings, and records test-set results in PROMPT_EVAL_RUN.

Entities

Entity	Module	Role in this feature
PROMPT_VERSION	AI, Matching & Evaluation	Prompt versions (v1, v2 ...) per generation type. The rule-based fallback is also recorded as a version.
PROMPT_EVAL_RUN	AI, Matching & Evaluation	Result of running a prompt version against a fixed test set (the required delivery evidence).
AI_GENERATION	AI, Matching & Evaluation	Log of every AI output (recommendation, project setup, pitch). Links an output to its prompt version.
AI_FEEDBACK	AI, Matching & Evaluation	User rating of an AI output: thumbs up/down + clarity 1-5 + optional reason.

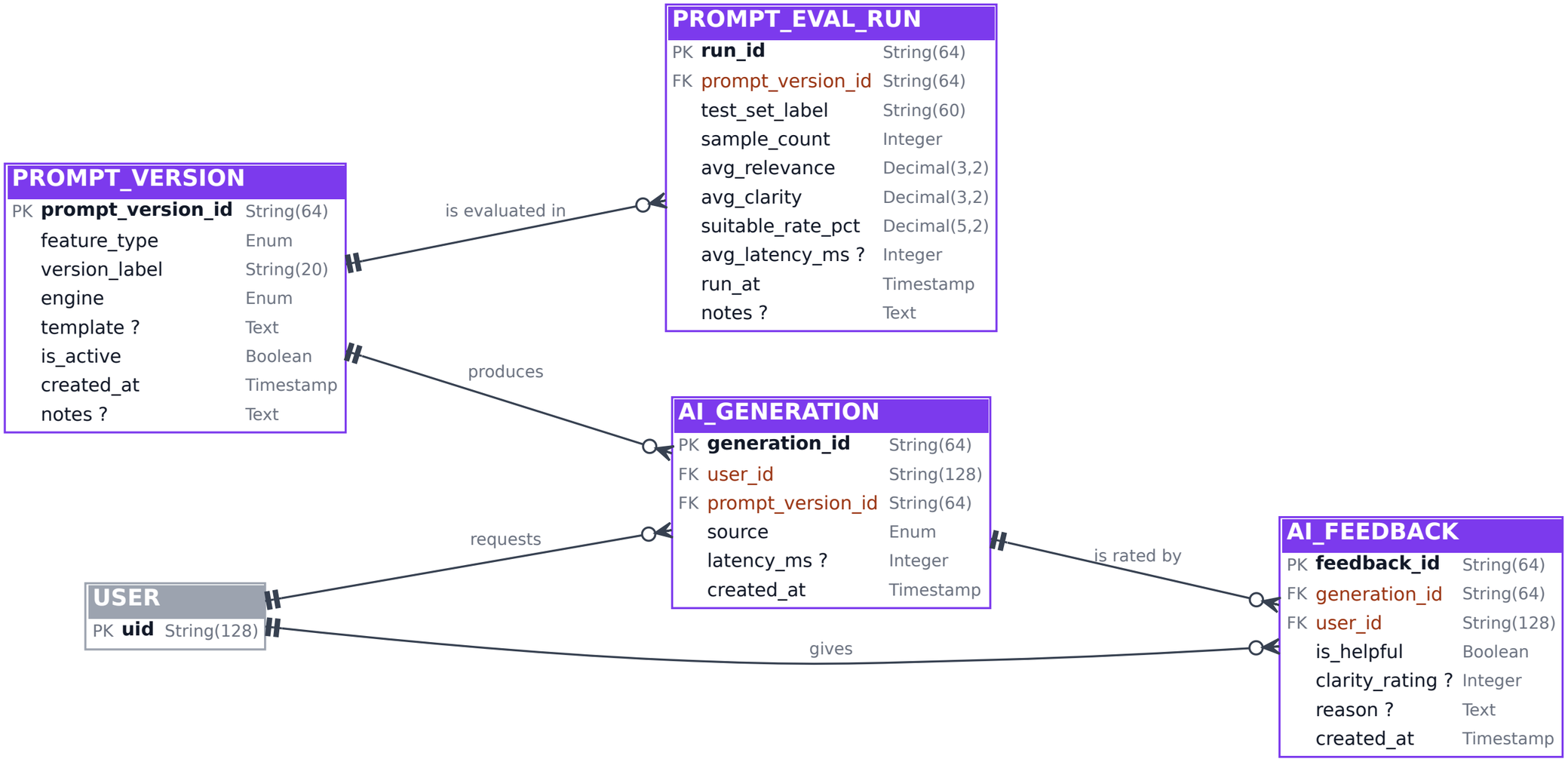
Relationships in this feature

Parent	Verb	Child	Cardinality	FK
PROMPT_VERSION	produces	**AI_GENERATION**	1:N (1 → 0..*)	`AI_GENERATION.prompt_version_id`
USER	requests	**AI_GENERATION**	1:N (1 → 0..*)	`AI_GENERATION.user_id`
PROMPT_VERSION	is evaluated in	**PROMPT_EVAL_RUN**	1:N (1 → 0..*)	`PROMPT_EVAL_RUN.prompt_version_id`
AI_GENERATION	is rated by	**AI_FEEDBACK**	1:N (1 → 0..*)	`AI_FEEDBACK.generation_id`
USER	gives	**AI_FEEDBACK**	1:N (1 → 0..*)	`AI_FEEDBACK.user_id`

Business rules from the documents

- Metrics: relevance and explanation clarity.
- Results are recorded in a table inside the documentation (the required delivery evidence).
- The best version is activated through is_active (Remote Config / Firestore config).
- Adaptive Matching is out of scope (future work).

ERD — Feature 12: Prompt Evaluation



8. Assumptions & Open Points

The documents leave some details open. These are the assumptions made instead of inventing facts; each should be confirmed with the team / supervisor before implementation.

- A Guest is not an entity: no account and no stored data; all permissions are read-only (Core §2).
- The Digital Twin is not a separate entity: it is treated as the skill profile = USER_SKILL (confirmed) + DETECTED_SKILL (inferred), following the practical proposal in section 4 of the Comprehensive Analysis.
- These entities are design additions needed to implement behaviour described in the documents but not named there: USER_DEVICE (FCM token), GITHUB_REPO and SKILL_EVIDENCE (repos and evidence), ROLE_SKILL (skills per role), AI_GENERATION (links an output to its prompt version), AI_USAGE_COUNTER (AI request cap - Risk R5).
- Enum values not given in the documents are proposals: project_type, difficulty (Beginner / Intermediate / Advanced), notification_type, feature_type, skill_kind. Level (Beginner / Junior / Intermediate / Advanced), task status and request status come from the documents.
- Project Purpose (Learning / Real_World) is mandatory, as proposed in decision point 4 (to become a formal FR). Its related fields (hours per week, min level, mentoring, end date) are nullable columns with a CHECK constraint per purpose, instead of two separate tables.
- A Join Request is allowed once per (project, applicant) for good: no re-applying after a rejection, because the document says 'once only, duplicates are not allowed'. If the team wants re-applying, drop the UNIQUE constraint and make it a partial UNIQUE on status = Pending.
- The owner is recorded as a PROJECT_MEMBER of type Owner so chat, tasks and notifications use the same logic.
- Save and Favorites = one entity, SAVED_PROJECT (per the notes in §6). Save, Express Interest and Request to Join are three separate entities.
- AI_FEEDBACK points to AI_GENERATION instead of directly to a recommendation, so the same feedback mechanism works for recommendations, pitches and project setup.
- Conversations: one team conversation per project. Uniqueness of a direct conversation between two users is enforced in the application layer (Cloud Function) because it is a multi-row rule.
- Free / Pro flavors are a build-time decision, not data, so there is no plan entity. If real payments are defined later, a SUBSCRIPTION entity would be added.
- The definition of Trending and the match-score weights are not approved in the documents (the weights are only an example), so they are not separate columns in the schema.
- Delete account removes all related rows (cascade). Transferring ownership of active projects is not defined in the documents.
- Behance is a link only (behance_url) with no data analysis (decision point 5). The GitHub token is not stored in the schema (NFR-07).
- Features 7 (Community) and 13 (Chatbot) are excluded from this scope, so no post, comment, like, tag, report or chatbot-message entities exist.
- Decision point 2 (merging My Projects and My Teams) does not affect the schema because both rely on PROJECT and PROJECT_MEMBER.